TECNOLÓGICO NACIONAL DE MÉXICO

ITESCAM

INSTITUTO TECNOLOGICO SUPERIOR DE CALKINI

INGENIERIA EN SISTEMAS COMPUTACIONALES

TOPICOS DE PROGRAMACIÓN MÓVIL

- **INGENIERA:** RUTH BETSAIDA MARTINEZ DOMINGUEZ

ALUMNO: TOMMY GAEL CAN MUT_8979

SEMESTRE:7°/

FECHA: MIERCOLES/04/NOVIEMBRE/2026

PRÁCTICA NO. 6 PERSISTENCIA LOCAL Y FUNCIONAMIENTO OFFLINE

-INTRODUCCIÓN

Esta práctica incorpora persistencia local y operación sin conexión. La aplicación conservará datos esenciales y podrá demostrar continuidad de uso cuando la red no esté disponible.

-OBJETIVO

Persistir preferencias simples y registros estructurados en almacenamiento local, implementando operaciones CRUD y una estrategia básica offline.

-LUGAR

Sala de cómputo / Aula de clases.

-SEMANA DE EJECUCIÓN

Semana 6

- MATERIAL Y EQUIPO

- Proyecto actualizado
- Paquetes sqflite y shared_preferences
- Dispositivo o emulador
- Git y GitHub

-DESARROLLO DE LA PRÁCTICA

Trabaje sobre una rama nueva y confirme que el proyecto compila antes de introducir cambios. Reproduzca el ejemplo guiado, comprenda cada bloque y después adapte el aprendizaje al proyecto integrador.

Preparación

```
flutter pub add shared_preferences
flutter pub add sqflite flutter
pub add path
```

```
stencia\flutter_application_1> flutter pub add shared_preferences
"shared_preferences" is already in "dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
  clock 1.1.2 (1.1.3 available)
  code_assets 1.2.1 (2.2.0 available)
  cupertino_icons 1.0.9 (2.0.0 available)
  hooks 2.0.2 (2.2.0 available)
  matcher 0.12.19 (0.12.20 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.17.0 (1.19.0 available)
  native_toolchain_c 0.19.2 (0.19.6 available)
  platform 3.1.6 (3.2.0 available)
  record_use 0.6.0 (1.1.1 available)
  shared_preferences_android 2.4.23 (2.5.4 available)
  shared_preferences_foundation 2.5.7 (2.6.1 available)
  sqflite 2.4.2+1 (2.4.6 available)
  sqflite_android 2.4.2+3 (2.4.6 available)
  sqflite_common 2.5.8 (2.5.17 available)
  sqflite_common_ffi 2.4.0+3 (2.4.6 available)
  sqflite_darwin 2.4.2 (2.4.6 available)
  sqflite_platform_interface 2.4.0 (2.4.4 available)
  sqlite3 3.5.2 (3.9.1 available)
  stack_trace 1.12.1 (1.12.2 available)
  synchronized 3.4.0+1 (3.6.1+1 available)
  test_api 0.7.10 (0.7.15 available)
  vector_math 2.2.0 (2.4.4 available)
  web 1.1.1 (1.1.2 available)
Got dependencies!
24 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
PS C:\Users\Gael\Desktop\C A N\Itescam 1 °\4to Año\Semestre 7\Extras\practica_06_persi
```

```
stencia\flutter_application_1>          flutter pub add path
"path" is already in "dependencies". Will try to update the constraint.
Resolving dependencies...
Downloading packages...
  clock 1.1.2 (1.1.3 available)
  code_assets 1.2.1 (2.2.0 available)
  cupertino_icons 1.0.9 (2.0.0 available)
  hooks 2.0.2 (2.2.0 available)
  matcher 0.12.19 (0.12.20 available)
  material_color_utilities 0.13.0 (0.13.1 available)
  meta 1.17.0 (1.19.0 available)
  native_toolchain_c 0.19.2 (0.19.6 available)
  platform 3.1.6 (3.2.0 available)
  record_use 0.6.0 (1.1.1 available)
  shared_preferences_android 2.4.23 (2.5.4 available)
  shared_preferences_foundation 2.5.7 (2.6.1 available)
  sqflite 2.4.2+1 (2.4.6 available)
  sqflite_android 2.4.2+3 (2.4.6 available)
  sqflite_common 2.5.8 (2.5.17 available)
  sqflite_common_ffi 2.4.0+3 (2.4.6 available)
  sqflite_darwin 2.4.2 (2.4.6 available)
  sqflite_platform_interface 2.4.0 (2.4.4 available)
  sqlite3 3.5.2 (3.9.1 available)
  stack_trace 1.12.1 (1.12.2 available)
  synchronized 3.4.0+1 (3.6.1+1 available)
  test_api 0.7.10 (0.7.15 available)
  vector_math 2.2.0 (2.4.4 available)
  web 1.1.1 (1.1.2 available)
Got dependencies!
24 packages have newer versions incompatible with dependency constraints.
Try `flutter pub outdated` for more information.
PS C:\Users\Gael\Desktop\C A N\Itescam 1 º\4to Año\Semestre 7\Extras\practica_06_persi
stencia\flutter_application_1>
```

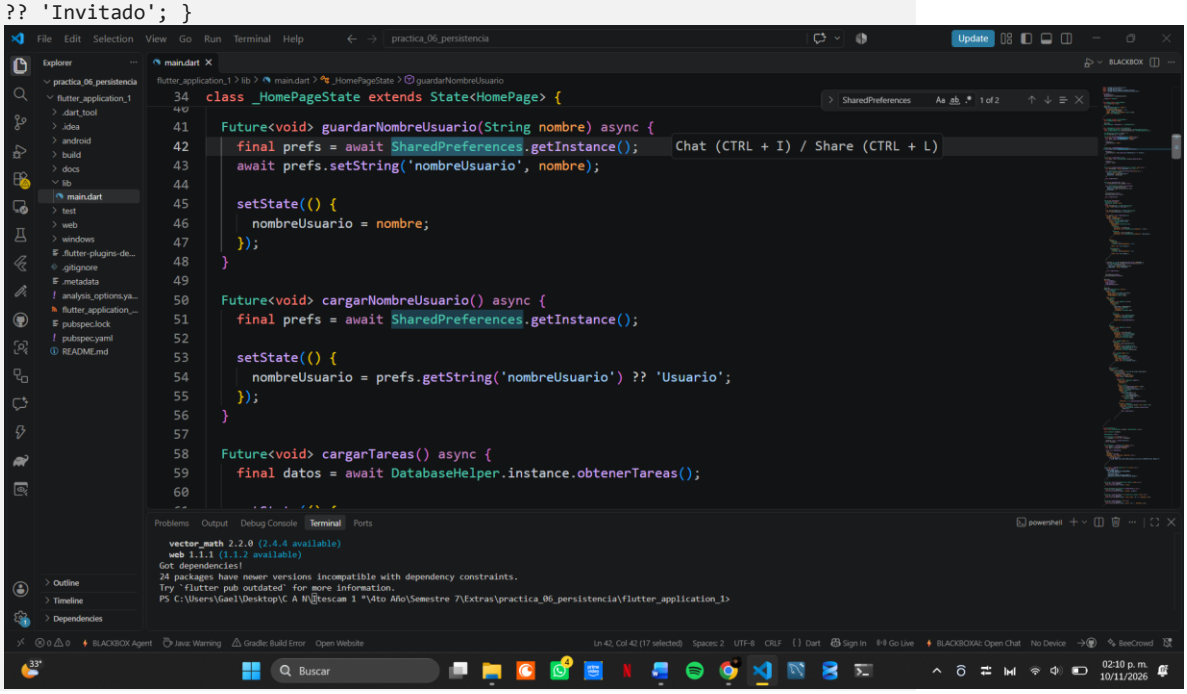
Parte A: Preferencia Simple

```
import 'package:shared_preferences/shared_preferences.dart';
Future<void> guardarNombre(String nombre) async {
final prefs = await SharedPreferences.getInstance();

  await prefs.setString('nombre_usuario', nombre);
}

Future<String> leerNombre() async {   final prefs = await
SharedPreferences.getInstance();   return prefs.getString('nombre_usuario')
```

```
?? 'Invitado'; }
```

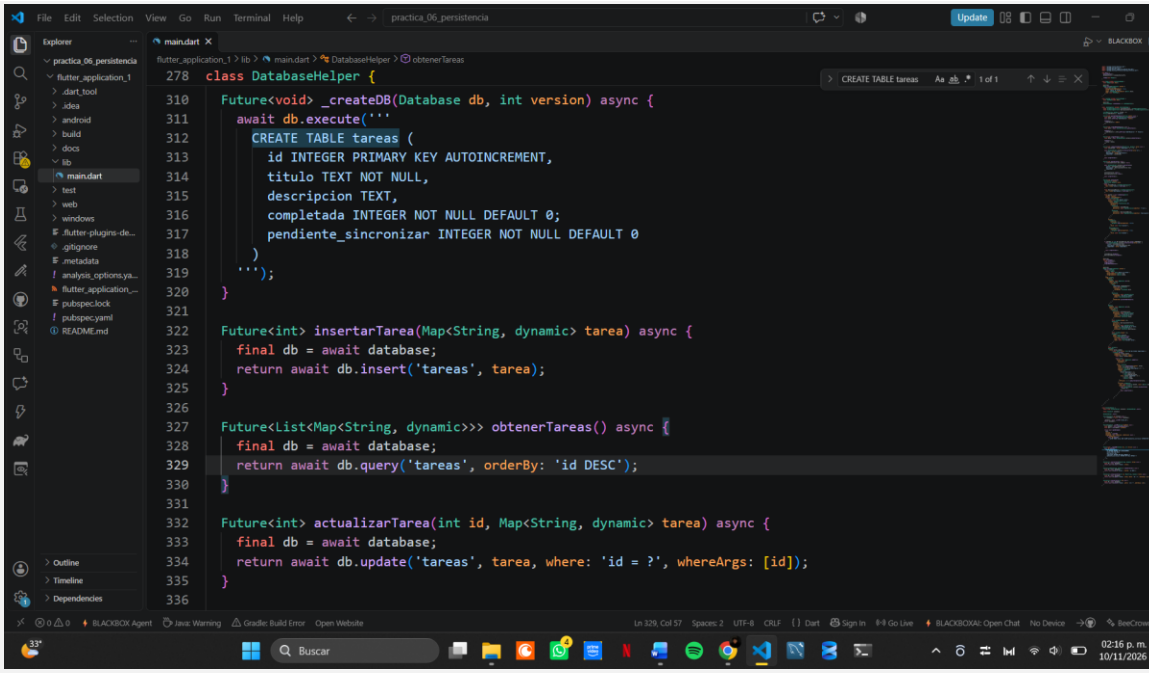


Parte B: Base Sqlite

Crea lib/database/db.dart con una clase de acceso:

```
import 'package:path/path.dart'; import
'package:sqflite/sqflite.dart';
 class DB {
  static Database? _db;

  static Future<Database> get database async {
if (_db != null) return _db!;
    final ruta = join(await getDatabasesPath(), 'notas.db');
    _db = await openDatabase(
ruta,       version: 1,
      onCreate: (db, version) => db.execute(
        'CREATE TABLE notas(id INTEGER PRIMARY KEY AUTOINCREMENT, titulo TEXT, contenido
TEXT)'
      ),
);
    return _db!;
  }
  static Future<int> insertar(String titulo, String contenido) async {
final db = await database;
    return db.insert('notas', {'titulo':titulo, 'contenido':contenido});
}
  static Future<List<Map<String,dynamic>>> listar() async {
final db = await database;
    return db.query('notas', orderBy:'id DESC');
  }    static Future<int> actualizar(int id, String titulo, String contenido)
async {     final db = await database;     return db.update('notas',
      {'titulo':titulo,'contenido':contenido},
where:'id=?', whereArgs:[id]);   }    static
Future<int> eliminar(int id) async {     final
db = await database;
    return db.delete('notas', where:'id=?', whereArgs:[id]);   }
}
```



```dart
class DatabaseHelper {
  Future<void> _createDB(Database db, int version) async {
    await db.execute('''
      CREATE TABLE tareas (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        titulo TEXT NOT NULL,
        descripcion TEXT,
        completada INTEGER NOT NULL DEFAULT 0;
        pendiente_sincronizar INTEGER NOT NULL DEFAULT 0
      )
    ''');
  }

  Future<int> insertarTarea(Map<String, dynamic> tarea) async {
    final db = await database;
    return await db.insert('tareas', tarea);
  }

  Future<List<Map<String, dynamic>>> obtenerTareas() async {
    final db = await database;
    return await db.query('tareas', orderBy: 'id DESC');
  }

  Future<int> actualizarTarea(int id, Map<String, dynamic> tarea) async {
    final db = await database;
    return await db.update('tareas', tarea, where: 'id = ?', whereArgs: [id]);
  }
```

Actividad Guiada

- Construye una pantalla con dos TextField y botones Guardar/Actualizar.
- Muestra DB.listar() en ListView.
- Al tocar una nota carga sus datos para editar.
- Agrega botón eliminar con diálogo de confirmación.
- Guarda con SharedPreferences el nombre del usuario y muéstralo al iniciar.
- Activa modo avión y verifica que las notas locales continúan disponibles.
- Diseña una bandera pendiente_sincronizar para discutir cómo identificar cambios locales aún no enviados a un servidor.

Vinculación con el proyecto integrador

Persistir la entidad principal y una preferencia de usuario; demostrar lectura, alta, actualización, baja y operación sin conexión.

Criterios de aceptación del avance:

- El CRUD local funciona
- Los datos permanecen después de reiniciar
- La preferencia de usuario se conserva

Entrega en Github

La práctica 6 debe publicarse en la rama practica-06-persistencia. Antes de editar, el equipo debe partir de develop actualizado. No deben subirse .dart_tool, build, credenciales, llaves ni archivos generados.

```
git checkout develop
git pull origin develop
git checkout -b practica-06-
persistencia # desarrollar y comprobar
la práctica dart format . flutter
analyze flutter test git add .
git commit -m "feat: completa practica 06"
git push -u origin practica-06-persistencia
```

Después del push, crear una pull request hacia develop. Incluir en la descripción: propósito, funciones agregadas, pasos para ejecutar, pruebas realizadas, capturas y problemas pendientes.

Evidencia de entrega	Requisito
Código fuente	Proyecto completo y ejecutable en la rama de la práctica
README.md	Objetivo, requisitos, instalación, ejecución y participantes
docs/evidencias/practica-06/	Capturas legibles y, cuando proceda, GIF o video breve mediante enlace
Pruebas	Resultado de flutter analyze, flutter test y casos manuales
Evidencia de entrega	**Requisito**
Enlaces para Moodle o plataforma	URL de pull request y URL del commit final

Condición de entrega: si el enlace no permite acceder a la versión solicitada o el código no corresponde al commit indicado, la práctica queda sin evidencia verificable hasta que se corrija dentro del plazo autorizado.

- EVALUACIÓN Y RESULTADOS

Ejercicio evaluable.

Desarrolla un gestor offline de tareas con SQLite: crear, consultar, editar, eliminar y marcar completada. Usa SharedPreferences para recordar una preferencia de interfaz (por ejemplo, filtro seleccionado o nombre). La aplicación debe funcionar sin Internet.

El resultado debe incluir el ejemplo guiado funcionando, el avance del proyecto integrador, las evidencias solicitadas y los enlaces verificables de GitHub. La rúbrica se aplica sobre el commit final declarado por el equipo.

Criterio	Descripción	Puntos
SQLite CRUD	Altas, consultas, cambios y bajas	35
SharedPreferences	Preferencia persistente	15
Offline	Operación local demostrada	15
UI/validación	Interfaz y reglas	20
Código/evidencias y GitHub	Organización y pruebas. Incluye README, evidencias, pull request y commit final verificables.	15

-REFERENCIAS

Flutter. Persistence cookbook. https://docs.flutter.dev/cookbook/persistence pub.dev.
sqflite. https://pub.dev/packages/sqflite
pub.dev. shared_preferences. https://pub.dev/packages/shared_preferences

-ANEXOS

- Código fuente y archivos de configuración en el repositorio de GitHub.
- Evidencias en docs/evidencias/practica-06/.
- Reporte de práctica elaborado con la estructura del Anexo A.

Capturas de la aplicación de escritorio para Windows llamada Gestor de tareas.

